

“EstateLy – Real Estate Property Discovery and Visit Scheduling System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

B Sree Nikhil Reddy	2520090058
CH Vihamsh	2520090090
V Vishwa Teja	2520090033

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

KLH (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“EstateLy – Real Estate Property Discovery and Visit Scheduling System”** is a record of bonafide work of **B Sree Nikhil Reddy – 2520090058**, **CH Vihamsh – 2520090090**, **V Vishwa Teja – 2520090033** submitted in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 2, during **2026–2027, Trimester 4**, at KLH (Deemed to be) University. The results embodied in this report have not been copied from any other student, department, or institution.

B Sree Nikhil Reddy – 2520090058
CH Vihamsh – 2520090090
V Vishwa Teja -- 2520090033

KLH (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“EstateLy – Real Estate Property Discovery and Visit Scheduling System”** is a bonafide work done and submitted by **B Sree Nikhil Reddy – 2520090058, CH Vihamsh – 2520090090 , V Vishwa Teja – 2520090033** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 2, at KLH (Deemed to be) University during **2026–2027, Trimester 4**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help, guidance, and encouragement of many people. I express my sincere thanks to all those who have assisted me, in one way or another, during the development of this project.

I express my deepest gratitude to my project guide, Divya Priya Degala, Assistant Professor, Department of Computer Science and Information Technology, for the valuable guidance, constant support, and encouragement provided throughout the development of this project. I also thank my Course Coordinator, Dr.R.Sateesh Kumar, for the support and direction given during the course.

I am grateful to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing the facilities and resources that enabled me to complete this project.

I thank all the teaching and non-teaching staff members who have assisted me directly or indirectly in the successful completion of this project. Finally, I sincerely thank my parents, friends, and classmates for their kind help and cooperation during my work.

B Sree Nikhil Reddy – 2520090058

CH Vihamsh – 2520090090

V Vishwa Teja -- 2520090033

Abstract

EstateLy is a web-based real estate platform designed to make property discovery, property saving, and visit scheduling easier through a single application. Searching for a property usually involves browsing many listings, keeping track of shortlisted properties manually, and arranging visits separately. These activities are often scattered across different sources, which makes it difficult for a user to keep track of what has been seen, what has been shortlisted, and when a visit is planned.

The proposed system provides a user-friendly web interface through which users can register and log in, browse the available properties, search the properties available inside the EstateLy application, and open detailed information for any property. Users can save properties to a wishlist, remove them when they are no longer needed, and schedule a visit by selecting a date and time. When a logged-in user opens a property details page, the view is recorded, and the Dashboard presents saved properties, upcoming visits, and the number of properties viewed, together with profile and settings options. Upcoming visits can be selected to display additional location information and a map.

The application uses a React and Vite frontend with React Router, Lucide React, and CSS; a Node.js and Express.js REST API backend; and a MySQL database named estateLy. The database consists of five tables – users, properties, favorites, visits, and property_views – in which the favorites, visits, and property_views tables relate each user to the properties they interact with. Passwords are handled using bcryptjs, authentication is performed using JSON Web Tokens (JWT), and the login session information is maintained on the frontend using localStorage. The demonstration data consists of three Hyderabad properties located in Gachibowli, Hitech City, and Kondapur, and the map locations used in the demonstration are fictional demonstration coordinates rather than exact property addresses. The system achieves its aim of providing a centralized and easy-to-use platform for property discovery, wishlist management, and visit scheduling.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	8
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	10
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Authentication and Security 3.5 Version Control and Development Tools	11
4	System Architecture	4.1 High-Level Architecture 4.2 Description of Each Layer 4.3 Application Workflow 4.4 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 UI/UX Design 5.4 Design Decisions	14
6	Implementation	6.1 Module-wise Implementation 6.2 Frontend Implementation 6.3 Backend Implementation 6.4 API Integration 6.5 Authentication 6.6 Database Integration	17
7	Features	7.1 Main Features 7.2 Home Page 7.3 Property Search 7.4 Property Details 7.5 Wishlist 7.6 Schedule Visit 7.7 Dashboard 7.8 Authentication	19
8	Testing	8.1 API Testing 8.2 Integration Testing 8.3 Frontend Testing 8.4 Database Testing 8.5 Test Cases and Results	23

S.No.	Chapters	Topics	Page No.
9	Deployment	9.1 Steps to Run the Project 9.2 Environment Configuration 9.3 Frontend and Backend Execution 9.4 Database Setup	25
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations	26
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations and Improvements	27
12	Conclusion	12.1 Summary 12.2 What Was Achieved 12.3 Skills Learned	28
13	References	References – Documentation, tools, and resources used	29
14	Appendices	Appendix A – Important Code Snippets Appendix B – Installation / Setup Instructions Appendix C – API Reference Appendix D – Screenshots	30

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Buying or renting a property involves comparing many options across different locations, prices, sizes, and property types. In most cases, a person searching for a property browses listings on several platforms, notes down the details manually, and contacts sellers or agents separately to arrange a visit. This works reasonably well for a small number of properties, but it becomes difficult to manage when a user has viewed many properties and needs to remember which ones were shortlisted and when each visit is planned.

EstateLy was developed to bring these activities together in a single web application. Users can browse the available properties, search them, open detailed information for a property, save properties to a wishlist, schedule visits by selecting a date and time, and review their own activity from a personal Dashboard. The application is built using a React and Vite frontend, a Node.js and Express.js backend exposing a REST API, and a MySQL database that stores users, properties, favorites, visits, and property views.

1.2 Problem Statement

Property seekers usually have to depend on several separate sources to discover properties, keep track of the ones they like, and organize site visits. There is no single place where a user can see the available properties, keep a personal shortlist, schedule a visit for a chosen date and time, and review which properties they have already viewed. As a result, useful information is scattered, shortlisted properties are easily forgotten, and planned visits are difficult to keep track of.

The problem addressed by this project is to design and develop a web application in which a registered user can discover and search the available properties, view their details, save and remove properties from a wishlist, schedule property visits, and view a summary of their saved properties, upcoming visits, and properties viewed in one place.

1.3 Scope of the Project

The project covers user registration and login; browsing the available properties; searching the properties available inside the EstateLy application; viewing detailed information for a property; saving properties to a wishlist and removing them; recording the properties a logged-in user has viewed; scheduling a property visit by selecting a date and time; and a Dashboard that displays saved properties, upcoming visits, the number of properties viewed, profile, and settings. Upcoming visits can be selected to display additional location information and a map.

The demonstration data consists of three properties located in Hyderabad (Gachibowli, Hitech City, and Kondapur). The map locations used in the demonstration are fictional demonstration coordinates and are not the exact addresses of the properties. The current version does not include online payments, property purchase transactions, or a separate interface for adding or managing property listings.

1.4 Objectives

- To provide a single web application in which users can browse the available properties.
- To allow users to search the properties available inside the EstateLy application.
- To display detailed information for each property, including its title, location, price, bedrooms, bathrooms, area, type, image, and description.
- To allow users to save properties to a wishlist and remove them from the wishlist.
- To allow users to schedule a property visit by selecting a date and time, and to view their upcoming visits.

- To record the properties viewed by a logged-in user and display the number of properties viewed on the Dashboard.
- To provide secure registration and login using bcryptjs and JSON Web Tokens (JWT).
- To store all application data in a MySQL relational database with clearly defined relationships between tables.

1.5 Importance of the Application

By bringing property discovery, wishlist management, and visit scheduling into one application, EstateLy reduces the effort a user spends moving between different sources and notes. The wishlist lets a user return to shortlisted properties at any time, the scheduled visits are stored against the user's account and displayed as upcoming visits, and the Dashboard gives a quick summary of the user's activity. Displaying location information and a map for an upcoming visit also helps the user prepare for the visit.

From an academic point of view, the project demonstrates the complete development of a three-tier full-stack web application, including a React frontend, a REST API built with Node.js and Express.js, a relational MySQL database, and token-based authentication.

1.6 Target Users / Audience

The primary users of EstateLy are individuals who are searching for properties and want to browse available properties, shortlist the ones they like, and schedule visits. Each user registers an account and logs in to access personalized features such as the wishlist, scheduled visits, and the Dashboard. The project is also intended for students, evaluators, and developers who wish to study the design and implementation of a React, Node.js, Express.js, and MySQL web application.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

EstateLy is a lightweight web application. The frontend development server, the Node.js backend, and the MySQL database can all be run on a single computer. The following configuration is sufficient for development and demonstration.

Table 2.1: Hardware requirements

Component	Requirement
Processor	Dual-core processor or better
RAM	4 GB minimum (8 GB recommended)
Storage	Approximately 1 GB of free disk space for Node.js modules, MySQL data, and project files
Display	Standard monitor capable of displaying a web browser
Network	Internet connection for installing project dependencies

2.2 Software Requirements

Table 2.2: Software requirements

Software	Purpose
Operating System	Windows, macOS, or Linux
Node.js and npm	Runtime for the backend and for running the frontend development tools
MySQL Server	Relational database server hosting the estateLy database
MySQL Workbench	Creating the database and tables and inspecting stored data
Web Browser	Google Chrome, Microsoft Edge, or Mozilla Firefox for running the application
Git	Version control

2.3 Development Tools and Frameworks

React.js, bootstrapped with Vite, was used to build the frontend, with React Router for navigation between pages, Lucide React for icons, and CSS for styling. Node.js with the Express.js framework was used to build the backend REST API. MySQL was used as the relational database and was managed using MySQL Workbench. Visual Studio Code was used as the code editor, a web browser was used to run and test the application, and Git with GitHub was used for version control and source code hosting.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a single-page application built with React.js and bundled and served during development using Vite. React divides the user interface into reusable components, and Vite provides a fast development server and build tooling. React Router handles client-side navigation between the Home, Properties, Property Details, Wishlist, Dashboard, Login, Register, and About pages without reloading the whole page. Lucide React provides the icons used in the interface, and the layout and appearance of the application are defined using plain CSS.

3.2 Backend

The backend is a REST API built with Node.js and Express.js. Express.js defines the routes for authentication, properties, favorites, visits, and property views. Each route receives the request from the frontend, interacts with the MySQL database, and returns the result to the frontend in JSON format.

3.3 Database

MySQL is used as the relational database, and MySQL Workbench was used to create and inspect the database. The database is named *estately* and consists of five tables: *users*, *properties*, *favorites*, *visits*, and *property_views*. The *favorites*, *visits*, and *property_views* tables store the *user_id* and *property_id* of the related user and property, which links user activity to the properties stored in the *properties* table.

3.4 Authentication and Security

Passwords are handled using *bcryptjs*, which hashes a password so that the plain-text password is not stored in the database. Authentication is performed using JSON Web Tokens (JWT): when a user logs in successfully, a token is issued to identify the user. On the frontend, the user information and authentication token are maintained using the browser's *localStorage* so that the login session is retained while the user navigates between pages. When the user logs out, this stored session information is cleared.

3.5 Version Control and Development Tools

Git and GitHub were used to track changes to the source code and to host the project repository. Visual Studio Code was used as the main editor for both the frontend and the backend. MySQL Workbench was used to design and verify the database, and a web browser was used to run the application and test the frontend and the API behavior.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture

EstateLy follows a three-tier architecture that separates the user interface, the application logic, and the stored data. Figure 4.1 shows the three layers and the way requests and responses flow between them.

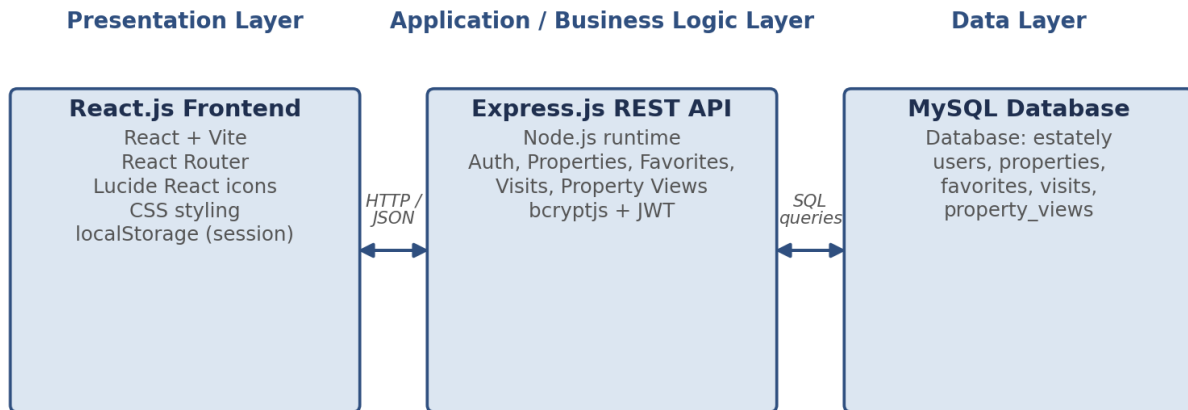


Figure 4.1: High-level three-tier architecture of EstateLy

4.2 Description of Each Layer

Presentation Layer. The Presentation Layer is the React and Vite single-page application that runs in the user's browser. It displays the pages of the application, collects user input such as login details, search text, and the visit date and time, and sends requests to the backend REST API. It does not communicate with the MySQL database directly. The login session information (user details and token) is kept in localStorage.

Application Layer. The Application Layer is the Node.js and Express.js REST API. It receives requests from the frontend, handles authentication using bcryptjs and JWT, and performs the operations for properties, favorites, visits, and property views by running queries on the database. The results are returned to the frontend as JSON responses.

Data Layer. The Data Layer is the MySQL database named estateLy. It stores the users, properties, favorites, visits, and property views of the application and maintains the relationships between them through the user_id and property_id columns.

4.3 Application Workflow

Figure 4.2 shows the overall workflow of the application. A user opens the Home page, browses or searches the properties, and opens a property details page. If the user is logged in, the property view is recorded. The logged-in user can then save the property to the wishlist or schedule a visit, and can review all activity on the Dashboard. A user who is not logged in is directed to log in or register first.

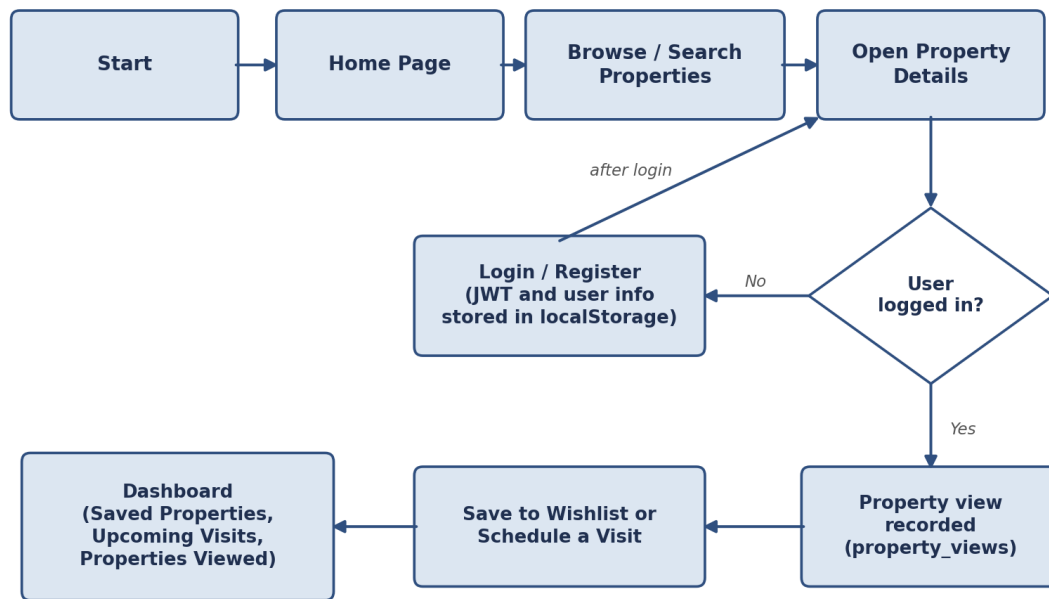


Figure 4.2: Application workflow of EstateLy

Figure 4.3 shows the interaction between the user, the frontend, the backend, and the database for one of the main features – scheduling a visit and then viewing it as an upcoming visit on the Dashboard. The same request and response pattern is followed by the other features such as the wishlist and property views.

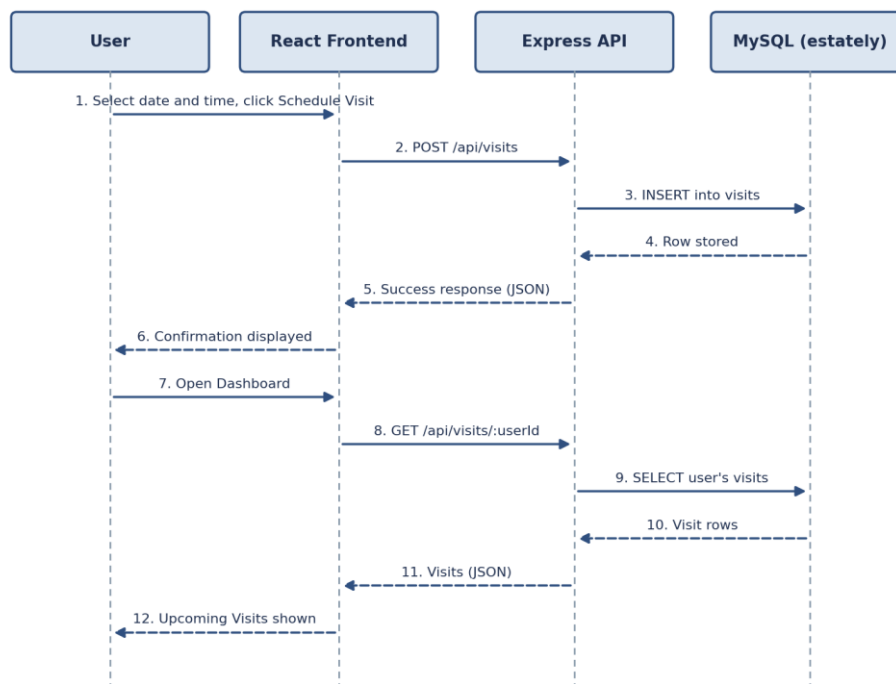


Figure 4.3: User interaction flow for scheduling and viewing a visit

4.4 Deployment Architecture

EstateLy has been developed and demonstrated in a local environment. The Vite development server serves the React application, the Express.js backend runs as a Node.js process, and the MySQL server hosts the estateLy database, all as separate processes on one computer. Deployment to a cloud or hosting platform was not performed; possible hosted deployment is discussed in Chapter 11.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The database was designed so that each kind of information is stored in its own table and every record can be identified uniquely through its id column. Information about users is stored only in the users table and information about properties is stored only in the properties table. The activity of a user – saving a property, scheduling a visit, or viewing a property – is stored in separate tables (favorites, visits, and property_views) that refer to the user and the property through user_id and property_id, instead of copying user or property details into every record. This avoids duplicated data and keeps the records consistent. Each table also contains a timestamp column (created_at, viewed_at) that records when the entry was created.

5.2 ER Diagram / Database Schema

The estate database consists of five tables, shown in Figure 5.1. Table 5.1 summarizes the purpose of each table, and Table 5.2 describes the relationships between them.

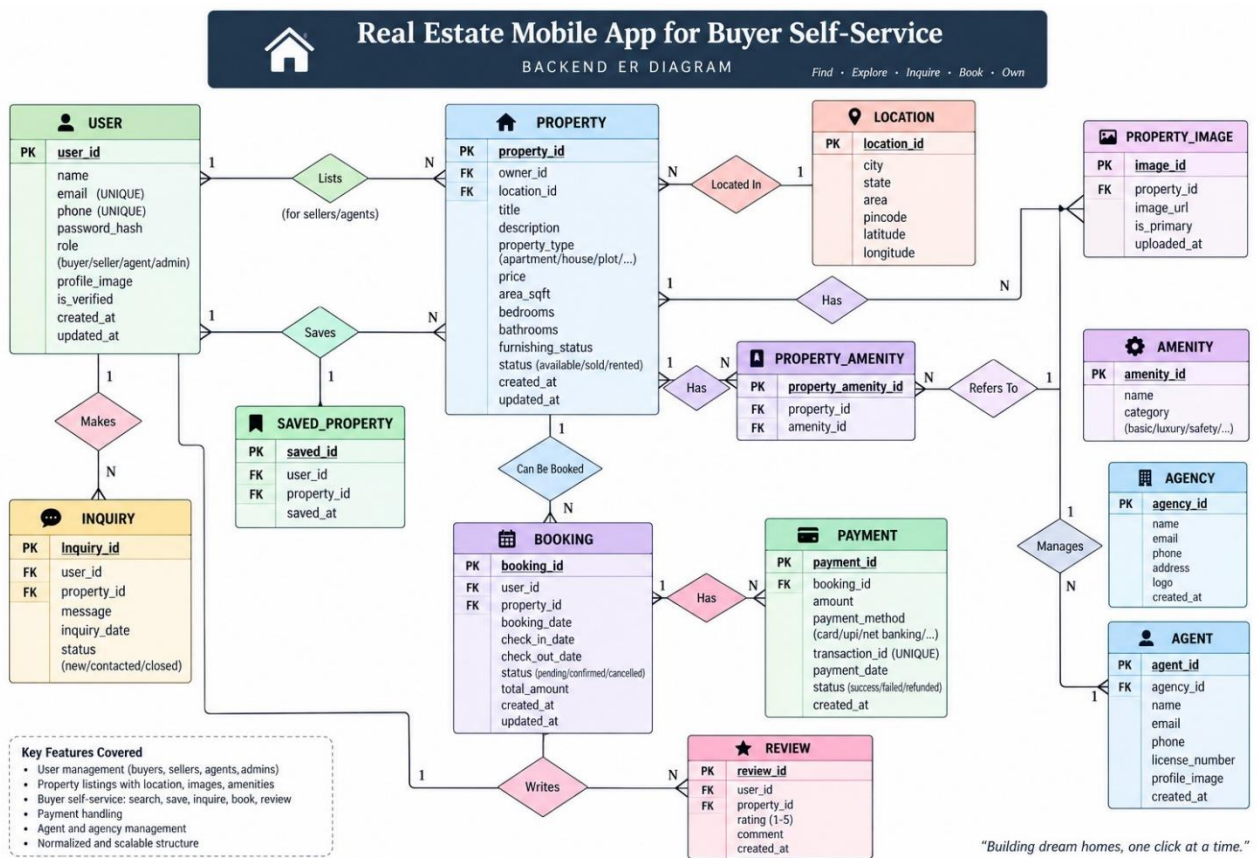


Figure 5.1: Entity-Relationship diagram of the estate database

Table 5.1: Tables of the estatefy database

Table	Purpose	Columns
users	Stores the registered users of the application	id, name, email, phone, password, created_at
properties	Stores the details of each available property	id, title, location, price, bedrooms, bathrooms, area, type, image, description, created_at
favorites	Stores the properties saved to a user's wishlist	id, user_id, property_id, created_at
visits	Stores the property visits scheduled by users	id, user_id, property_id, visit_date, visit_time, status, created_at
property_views	Stores each time a logged-in user opens a property details page	id, user_id, property_id, viewed_at

Table 5.2: Relationships between tables

Relationship	Type	Description
users – favorites	One-to-many	One user can save many properties to the wishlist
properties – favorites	One-to-many	One property can be saved by many users
users – visits	One-to-many	One user can schedule many visits
properties – visits	One-to-many	One property can have many visits scheduled for it
users – property_views	One-to-many	One user can view many properties
properties – property_views	One-to-many	One property can be viewed many times

Through the favorites, visits, and property_views tables, the users and properties tables are related as many-to-many: a user can save, visit, and view many properties, and a property can be saved, visited, and viewed by many users. Figure 5.2 shows the tables as created in MySQL Workbench.

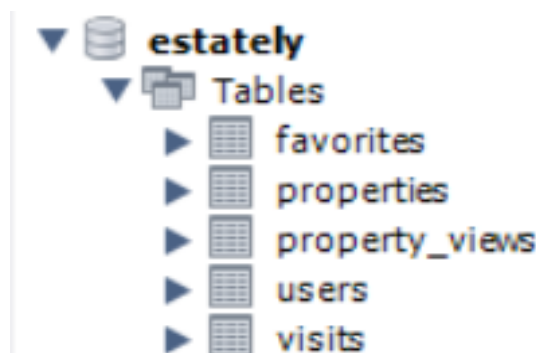


Figure 5.2: MySQL Database Tables (estatefy)

5.3 UI/UX Design

The user interface is organized as a set of pages connected using React Router. A consistent layout and CSS styling are used across the pages so that the user can move between them easily, and Lucide React icons are used to support the interface elements. The key interactions – searching properties, saving a property, and scheduling a visit by choosing a date and time – are available directly from the pages where they are needed. Table 5.3 lists the pages of the application.

Table 5.3: Pages of the EstateLy application

Page	Purpose
Home	Entry page of the application
Properties	Lists the available properties and provides search
Property Details	Displays the detailed information of a selected property; provides the options to save the property and schedule a visit
Wishlist	Displays the properties saved by the user and allows them to be removed
Dashboard	Displays saved properties, upcoming visits, properties viewed, profile, and settings
Login	Allows a registered user to log in
Register	Allows a new user to create an account
About	Provides information about the application

5.4 Design Decisions

- **Three-tier separation:** the frontend, backend, and database are kept separate, and the frontend communicates with the database only through the REST API.
- **Separate tables for user activity:** favorites, visits, and property_views are stored in their own tables so that wishlist items, scheduled visits, and viewed properties can be retrieved independently.
- **Property views for logged-in users only:** a view is recorded only when a logged-in user opens a property details page, so that the number of properties viewed belongs to a specific user.
- **Search within EstateLy:** the search feature searches only the properties available inside the EstateLy application; it is not an external search engine.
- **Session on the frontend:** the user information and authentication token are kept in localStorage so that the session continues while the user navigates between pages.
- **Demonstration locations:** the properties use Hyderabad locations (Gachibowli, Hitech City, and Kondapur), and the map locations are fictional demonstration coordinates, not exact property addresses.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The application is implemented as a set of modules, each of which combines frontend pages, backend API routes, and database tables. Table 6.1 summarizes the modules.

Table 6.1: Modules of EstateLy

Module	Description	API / Table
Authentication	Registration, login, and logout of users	POST /api/auth/register, POST /api/auth/login; users
Property Browsing and Search	Listing the available properties and searching within EstateLy	GET /api/properties; properties
Property Details and Views	Displaying a property and recording the view for a logged-in user	POST /api/property-views; property_views
Wishlist	Saving, viewing, and removing saved properties	POST, GET, DELETE /api/favorites; favorites
Schedule Visit	Selecting a date and time and storing the visit; retrieving upcoming visits	POST /api/visits, GET /api/visits/:userId; visits
Dashboard	Showing saved properties, upcoming visits, properties viewed, profile, and settings	GET APIs of favorites, visits, and property views

6.2 Frontend Implementation

The frontend is a React application created with Vite. Each page of the application (Home, Properties, Property Details, Wishlist, Dashboard, Login, Register, and About) is implemented as a React component, and React Router maps each page to its route so that navigation does not reload the whole application. The pages communicate with the backend by sending HTTP requests to the REST API and display the data returned in JSON format. Lucide React supplies the icons, and CSS is used for the styling of the pages. The user information and token received after login are stored in localStorage and are used to identify the logged-in user while using the application.

6.3 Backend Implementation

The backend is a Node.js application that uses the Express.js framework. Routes are defined for authentication, properties, favorites, visits, and property views. Each route receives the request, runs the required query on the MySQL database, and sends the result back to the frontend as a JSON response. The backend also performs the password handling with bcryptjs during registration and login, and issues a JWT after a successful login.

6.4 API Integration

The frontend uses the REST API endpoints listed in Table 6.2. Each endpoint serves a specific purpose in the application.

Table 6.2: REST API endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Registers a new user and stores the account in the users table
POST	/api/auth/login	Verifies the login details of a user and returns the authentication token
GET	/api/properties	Returns the available properties from the properties table
POST	/api/favorites	Saves a property to the user's wishlist (favorites table)
GET	/api/favorites/:userId	Returns the properties saved by the given user
DELETE	/api/favorites/:propertyId	Removes a saved property from the wishlist
POST	/api/visits	Schedules a visit by storing the selected property, date, and time (visits table)
GET	/api/visits/:userId	Returns the visits scheduled by the given user, used to display upcoming visits
POST	/api/property-views	Records that a logged-in user viewed a property (property_views table)
GET	/api/property-views/:userId/count	Returns the number of properties viewed by the given user for the Dashboard

6.5 Authentication

The authentication process works as follows:

1. **Registration:** the user enters the details on the Register page. The backend receives them at POST /api/auth/register, processes the password using bcryptjs, and stores the new user in the users table.
2. **Login:** the user enters the login details on the Login page. The backend verifies them at POST /api/auth/login and, if they are valid, returns a JSON Web Token (JWT).
3. **Session:** the frontend stores the user information and the token in localStorage so that the user remains logged in while navigating between pages.
4. **Use of the session:** the stored user information is used to identify the user when saving properties, scheduling visits, recording property views, and loading the Dashboard.
5. **Logout:** when the user logs out, the stored session information is removed from localStorage.

6.6 Database Integration

The backend connects to the MySQL database named `estately` and runs queries to read and write data in the five tables. User accounts are stored in `users`, available properties are read from `properties`, wishlist entries are stored in `favorites`, scheduled visits are stored in `visits`, and viewed properties are stored in `property_views`. Table 6.3 shows the property data used in the project.

Table 6.3: Property data used in the project

Title	Location	Price	Beds	Baths	Area	Type
Modern Luxury Villa	Gachibowli, Hyderabad	₹2.45 Cr	4	4	3,200 sq.ft	Villa
Premium City Apartment	Hitech City, Hyderabad	₹1.28 Cr	3	3	2,100 sq.ft	Apartment
Elegant Family Home	Kondapur, Hyderabad	₹1.85 Cr	3	3	2,650 sq.ft	House

CHAPTER 7: FEATURES

7.1 Main Features

- Register and log in, and log out securely.
- Browse the available properties.
- Search the properties available inside the EstateLy application.
- View detailed information of a property.
- Save properties to a wishlist and remove them from the wishlist.
- Schedule a property visit by selecting a date and time.
- View upcoming scheduled visits, with location information and a map.
- View saved properties.
- Track the properties viewed.
- Manage the user profile and settings from the Dashboard.

7.2 Home Page

The Home page is the entry page of EstateLy. It introduces the application and lets the user move to the other pages, such as Properties, Wishlist, Dashboard, Login, Register, and About, using React Router navigation.

7.3 Property Search

The Properties page displays the available properties retrieved from the backend through GET /api/properties. The search feature searches the properties available inside the EstateLy application using the text entered by the user. It is not Google search or any external search engine, and it does not return results from outside EstateLy.

7.4 Property Details

When the user opens a property, the Property Details page displays the detailed information stored for that property – title, location, price, bedrooms, bathrooms, area, type, image, and description. From this page, the user can save the property to the wishlist or schedule a visit. When a logged-in user opens this page, the view is recorded in the property_views table through POST /api/property-views.

7.5 Wishlist

A logged-in user can save a property to the wishlist. The saved property is stored in the favorites table through POST /api/favorites, and the saved properties of the user are retrieved through GET /api/favorites/:userId and displayed on the Wishlist page. A property that is no longer needed can be removed from the wishlist using DELETE /api/favorites/:propertyId.

7.6 Schedule Visit

The user selects a date and time for a property and submits the request. The visit is stored in the visits table through POST /api/visits along with the user, the property, the visit date, the visit time, and the visit status.

7.7 Dashboard

The Dashboard gives the user a summary of their activity. It displays the number of Saved Properties, the Upcoming Visits, and the number of Properties Viewed (obtained through GET /api/property-views/:userId/count), together with the saved property information, the upcoming visit information, the Profile, and the Settings. The upcoming visits are retrieved through GET /api/visits/:userId.

An upcoming visit can be selected to display additional location information and a map. The locations used are Gachibowli, Hitech City, and Kondapur in Hyderabad. The map locations used in the demonstration are fictional demonstration coordinates and do not represent the exact addresses of the properties.

7.8 Authentication

New users create an account on the Register page, and registered users log in on the Login page. After a successful login, the user information and token are maintained in localStorage, and the personalized features such as the wishlist, visit scheduling, and the Dashboard become available. The user can log out, which ends the session.

CHAPTER 8: TESTING

8.1 API Testing

The REST API endpoints listed in Table 6.2 are tested by sending requests to the backend and checking the responses and the resulting changes in the database. The API testing covers registration and login, retrieving properties, adding, retrieving, and removing favorites, creating and retrieving visits, and recording and counting property views. API testing tool used: Postman.

8.2 Integration Testing

Integration testing checks that the React frontend, the Express.js backend, and the MySQL database work correctly together. The complete user flow is tested from end to end: registering a user, logging in, browsing and searching properties, opening a property details page, saving and removing a property from the wishlist, scheduling a visit, and checking that the Dashboard shows the saved properties, upcoming visits, and the number of properties viewed.

8.3 Frontend Testing

Frontend testing is carried out manually in a web browser. Each page of the application is opened to confirm that it loads and displays the expected content, that navigation between pages works through React Router, that the forms (Register, Login, and Schedule Visit) accept input, and that the buttons for saving and removing properties and for logging out work as expected.

8.4 Database Testing

Database testing is carried out using MySQL Workbench. After each operation in the application, the related table is inspected to confirm the change: a new row in users after registration, a row in favorites when a property is saved and its removal when the property is removed, a row in visits when a visit is scheduled, and a row in property_views when a logged-in user opens a property details page.

8.5 Test Cases and Results

Table 8.1 lists the test cases for the main functionality of EstateLy. The Actual Result column is to be completed after executing the tests.

Table 8.1: Test cases

#	Test Case	Expected Result	Actual Result
1	Register with valid details	New user is created and stored in the users table	Passed – New user was successfully registered and stored in the database.
2	Login with valid credentials	Login succeeds; token and user information are stored in localStorage	Passed – Login was successful and user information/token were stored in localStorage.
3	Login with incorrect credentials	Login does not succeed and the user is informed	Passed – Invalid credentials were rejected and an error message was displayed.
4	Request GET /api/properties	The available properties are returned	Passed – The API successfully returned the available properties.
5	Search for a property	Matching properties from EstateLy are displayed	Passed – Matching EstateLy properties were displayed based on the search criteria.
6	Open a property details page	Details of the selected property are displayed	Passed – The selected property's details were displayed successfully.
7	Open a property details page while logged in	A row is added to the property_views table	Passed – The property view was recorded in the property_views table.
8	Save a property to the wishlist	A row is added to the favorites table and the property appears on the Wishlist page	Passed – The property was saved to favorites and appeared on the Wishlist page.
9	Remove a property from the wishlist	The property is removed from favorites and no longer appears on the Wishlist page	Passed – The property was removed from favorites and disappeared from the Wishlist page.
10	Schedule a visit with a date and time	A row is added to the visits table	Passed – The visit was successfully scheduled and stored in the visits table.
11	View upcoming visits on the Dashboard	The scheduled visits of the user are displayed	Passed – Upcoming scheduled visits were displayed correctly on the Dashboard.

#	Test Case	Expected Result	Actual Result
12	Select an upcoming visit	Additional location information and a map are displayed	Passed – Selecting a visit displayed the location details and map successfully.
13	Open the Dashboard	Saved Properties, Upcoming Visits, and Properties Viewed are displayed	Passed – Dashboard statistics and user information were displayed correctly.
14	Log out	Session information is removed and the user is logged out	Passed – The user was logged out and session information was removed.
15	Navigate between pages	Each page opens through React Router without errors	Passed – All configured pages opened successfully through React Router without navigation errors.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Run the Project

EstateLy is run locally for development and demonstration. The steps are:

1. Install Node.js (with npm), MySQL Server, and MySQL Workbench.
2. Obtain the project source code from the GitHub repository: [Insert repository URL].
3. Create the estateLy database and its five tables in MySQL (Section 9.4).
4. Configure the backend environment settings (Section 9.2).
5. Start the backend server and the frontend development server (Section 9.3).
6. Open the application in a web browser.

9.2 Environment Configuration

The backend requires the following configuration values: the MySQL connection details (host, user, password, and the database name estateLy), the secret used to sign JWT tokens, and the port on which the backend listens. The frontend needs the address of the backend API. These settings are kept in [Insert configuration file or method used, e.g., an environment file] and are set to match the local machine. Real passwords and secrets should not be shared in the report or committed to a public repository.

9.3 Frontend and Backend Execution

Both the backend and the frontend are started from their own project folders. The dependencies are installed with npm install in each folder.

Table 9.1: Commands to run the application

Part	Commands
Backend (Node.js + Express.js)	npm install, then cd backend, and npm run dev
Frontend (React + Vite)	npm install, then npm run dev

After both servers are running, the application is opened in the browser at the address shown by the Vite development server.

9.4 Database Setup

1. Open MySQL Workbench and connect to the local MySQL server.
2. Create the database named estateLy.
3. Create the five tables – users, properties, favorites, visits, and property_views – with the columns listed in Table 5.1 using the estateLy_database.sql script executed in MySQL Workbench.
4. Insert the three demonstration properties listed in Table 6.3 into the properties table.
5. Confirm that the tables are created by viewing them in MySQL Workbench (Figure 5.2).

CHAPTER 10: CHALLENGES AND LIMITATIONS

10.1 Issues Faced During Development

During the development of **EstateLy**, several technical and integration-related issues were encountered while building the frontend, backend, database, and user interaction features.

- Initially, connecting the React frontend with the Node.js/Express backend caused issues while sending and retrieving property, user, wishlist, and visit data.
- Problems were encountered while connecting the backend to MySQL and managing data across tables such as users, properties, favorites, visits, and property_views.
- The wishlist needed to be connected to the logged-in user so that saved properties were stored in MySQL and displayed correctly across the Home page, Wishlist page, and Dashboard.

10.2 Solutions Applied

The following solutions were applied to resolve the issues encountered during development:

- REST API endpoints were created using Node.js and Express, and the React frontend was connected to these endpoints using `fetch()` requests. This allowed the application to exchange property, user, wishlist, visit, and property-view data with the backend.
- A structured MySQL database named `estately` was created with separate tables for users, properties, favorites, visits, and property views. Foreign keys and unique constraints were used to maintain relationships and prevent duplicate records.
- The favorites API was implemented with POST, GET, and DELETE operations. The logged-in user's ID was stored in `localStorage`, allowing EstateLy to retrieve and update only that user's saved properties.

10.3 Current Limitations

- The properties are stored in the database; the current version does not provide an interface for adding, editing, or deleting property listings.
- A visit can be scheduled and its upcoming visits viewed, but rescheduling or cancelling a visit is not part of the current API.
- The search covers only the properties available inside EstateLy and does not search any external source.
- The map locations are fictional demonstration coordinates and do not represent exact property addresses.
- The demonstration uses a small data set of three properties.
- The application does not include online payments, property booking transactions, or email or SMS notifications.
- The application has been run in a local environment and has not been deployed to a cloud or hosting platform.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- An interface for property owners or administrators to add, edit, and remove property listings.
- Cancelling and rescheduling of scheduled visits.
- Email or SMS reminders for upcoming visits.
- Advanced filters in property search, such as price range, number of bedrooms, and property type.
- Multiple images for each property and a comparison view for shortlisted properties.
- Editing of the user profile and additional account settings.

11.2 Possible Integrations and Improvements

- Integrating a real map service so that exact property locations and directions can be shown instead of demonstration coordinates.
- Deploying the frontend, backend, and database to a hosting platform so that the application is accessible online.
- Storing the authentication token using a more secure mechanism than localStorage.
- Adding automated tests for the API and the frontend to complement manual testing.
- Using pagination and caching to handle a larger number of properties efficiently.
- Integrating a payment gateway if booking or token-amount payments are introduced.

CHAPTER 12: CONCLUSION

12.1 Summary

This project set out to bring property discovery, wishlist management, and visit scheduling into a single web application. EstateLy was developed with a React and Vite frontend, a Node.js and Express.js REST API, and a MySQL database named estateLy containing five related tables. Users can register, log in, browse and search properties, view property details, save and remove properties from a wishlist, schedule visits, and review their activity on a Dashboard.

12.2 What Was Achieved

- A working three-tier web application with a React frontend, an Express.js backend, and a MySQL database.
- User registration and login using bcryptjs and JWT, with the login session maintained in localStorage.
- Property browsing, search within EstateLy, and detailed property pages for properties in Gachibowli, Hitech City, and Kondapur.
- A wishlist stored in the favorites table, with the ability to remove saved properties.
- Recording of property views in the property_views table and display of the number of properties viewed.
- Visit scheduling stored in the visits table, with upcoming visits displayed on the Dashboard along with location information and a demonstration map.

12.3 Skills Learned

- Building a single-page application using React, Vite, React Router, Lucide React, and CSS.
- Designing and implementing a REST API using Node.js and Express.js.
- Designing a relational database in MySQL and relating tables through user_id and property_id.
- Implementing authentication using bcryptjs and JWT, and maintaining a login session using localStorage.
- Connecting a frontend, a backend, and a database into one working application.
- Using Git and GitHub for version control, and MySQL Workbench for database design and verification.

CHAPTER 13: REFERENCES

- React – Official Documentation, <https://react.dev>
- Vite – Official Documentation, <https://vitejs.dev>
- React Router – Official Documentation, <https://reactrouter.com>
- Lucide – Icon Library Documentation, <https://lucide.dev>
- Node.js – Official Documentation, <https://nodejs.org/en/docs>
- Express.js – Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- MySQL Workbench – Documentation, <https://dev.mysql.com/doc/workbench/en/>
- bcryptjs – npm documentation, <https://www.npmjs.com/package/bcryptjs>
- JSON Web Token Introduction, <https://jwt.io/introduction>
- MDN Web Docs – Window.localStorage, <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- Git – Official Documentation, <https://git-scm.com/doc>
- Visual Studio Code – Documentation, <https://code.visualstudio.com/docs>

CHAPTER 14: APPENDICES

Appendix A: Important Code Snippets

The following code snippets from the project are to be inserted by the student.

A.1 Registration with password hashing (bcryptjs)

```
const hashedPassword = await bcrypt.hash(password, 10);

await pool.query(
  "INSERT INTO users (name, email, phone, password) VALUES (?, ?, ?, ?)",
  [name, email, phone, hashedPassword]
);
```

A.2 Login and JWT generation

```
const [users] = await pool.query(
  "SELECT * FROM users WHERE email = ?",
  [email]
);

const valid = await bcrypt.compare(password, users[0].password);

const token = jwt.sign(
  { userId: users[0].id },
  process.env.JWT_SECRET
);
```

A.3 Saving a property to the wishlist (POST /api/favorites)

```
await pool.query(
  "INSERT INTO favorites (user_id, property_id) VALUES (?, ?)",
  [user_id, property_id]
);

res.status(201).json({
  message: "Property added to wishlist"
});
```

A.4 Scheduling a visit (POST /api/visits)

```
await pool.query(
  `INSERT INTO visits
  (user_id, property_id, visit_date, visit_time)
  VALUES (?, ?, ?, ?)`,
  [user_id, property_id, visit_date, visit_time]
);

res.status(201).json({
  message: "Visit scheduled successfully"
});
```

A.5 Recording a property view (POST /api/property-views)

```
await pool.query(
  `INSERT INTO property_views
    (user_id, property_id)
    VALUES (?, ?)
    ON DUPLICATE KEY UPDATE viewed_at = CURRENT_TIMESTAMP`,
  [user_id, property_id]
);

res.status(201).json({
  message: "Property view recorded"
});
```

Appendix B: Installation / Setup Instructions

- Install Node.js, MySQL Server, and MySQL Workbench.
- Clone or download the project from [Insert repository URL].
- Create the estatefy database and its tables in MySQL Workbench.
- In the backend folder, run npm install, configure the environment settings, and start the backend server.
- In the frontend folder, run npm install and then npm run dev.
- Open the address shown by the Vite development server in a web browser.

Appendix C: API Reference

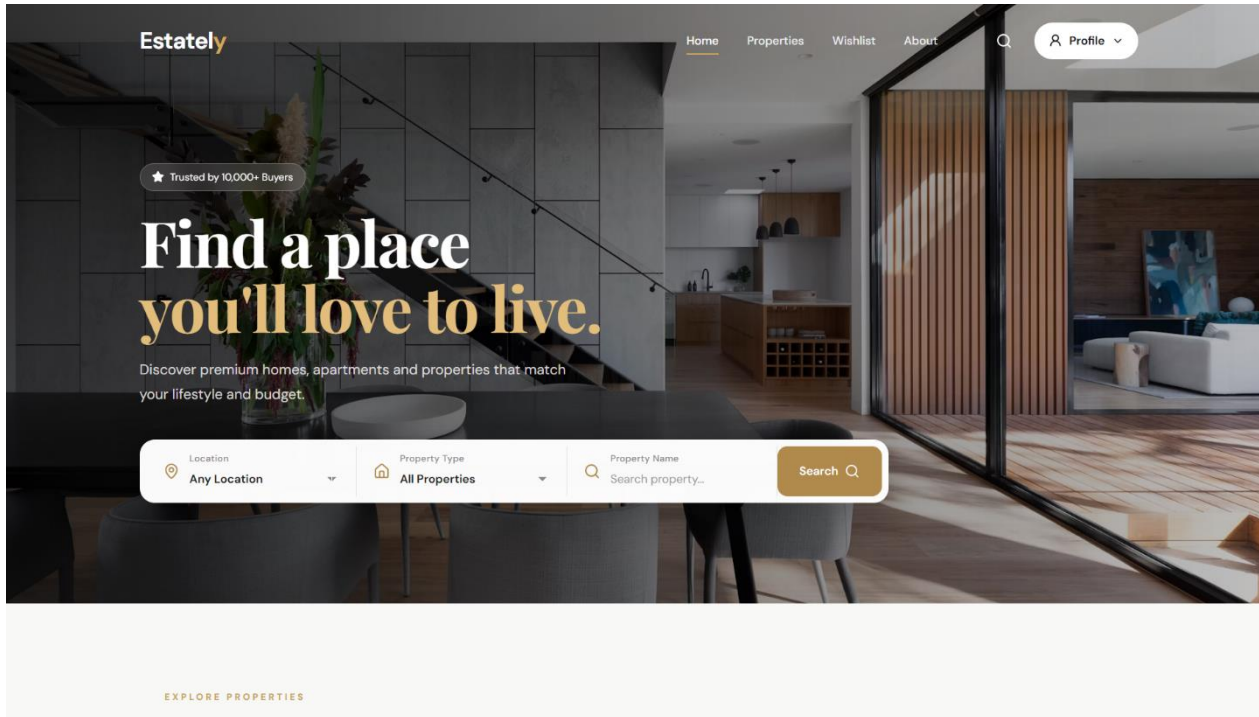
Table C.1: API reference

Resource	Method	Endpoint	Description
Authentication	POST	/api/auth/register	Register a new user
Authentication	POST	/api/auth/login	Log in and obtain a token
Properties	GET	/api/properties	Get all available properties
Favorites	POST	/api/favorites	Save a property to the wishlist
Favorites	GET	/api/favorites/:userId	Get a user's saved properties
Favorites	DELETE	/api/favorites/:propertyId	Remove a saved property
Visits	POST	/api/visits	Schedule a visit
Visits	GET	/api/visits/:userId	Get a user's scheduled visits
Property Views	POST	/api/property-views	Record a property view
Property Views	GET	/api/property-views/:userId/count	Get a user's viewed property count

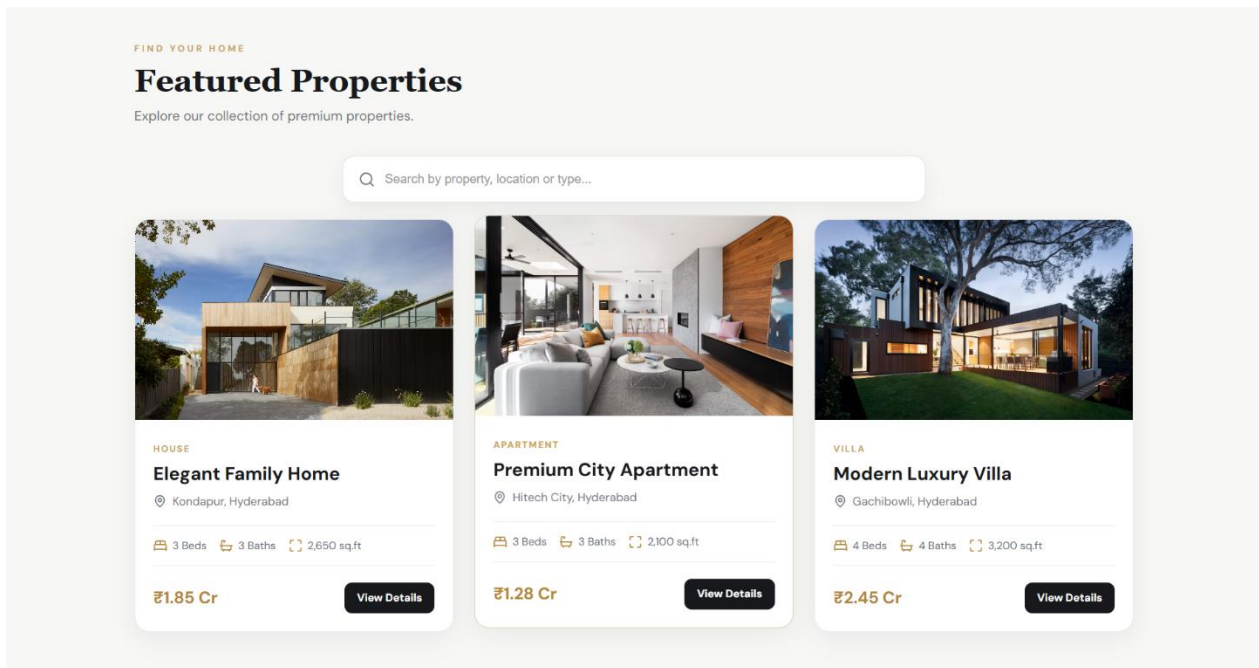
Appendix D: Screenshots

The following screenshots are to be inserted from the running application before submission:

1. Home Page



2. Properties Page



3. Property Details Page



VERIFIED PROPERTY

Elegant Family Home

📍 Kondapur, Hyderabad

₹1.85 Cr

 **3** Bedrooms

 **3** Bathrooms

 **2,650 sq.ft** Built-up Area

 **AR**

PROPERTY CONSULTANT

Arjun Reddy

EstateLy Verified Agent

Schedule a Visit

Contact Agent

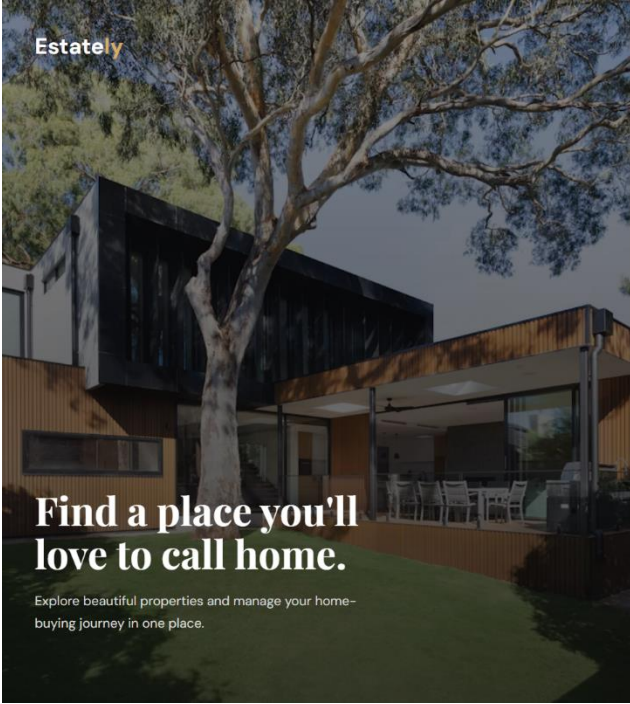
 Your information is safe and will only be shared with the property consultant.

About this property

Experience comfortable and modern living in this beautiful property located in one of the most desirable areas of Hyderabad. Designed with spacious rooms, quality interiors and convenient access to essential facilities.

The property offers an excellent combination of comfort, location and lifestyle, making it suitable for families looking for a premium home.

4. Login Page



EstateLy

Find a place you'll love to call home.

Explore beautiful properties and manage your home-buying journey in one place.

WELCOME BACK

Sign in to EstateLy

Access your saved properties and scheduled visits.

Email Address

 you@example.com

Password

 Enter your password

☐ Remember me

[Forgot Password?](#)

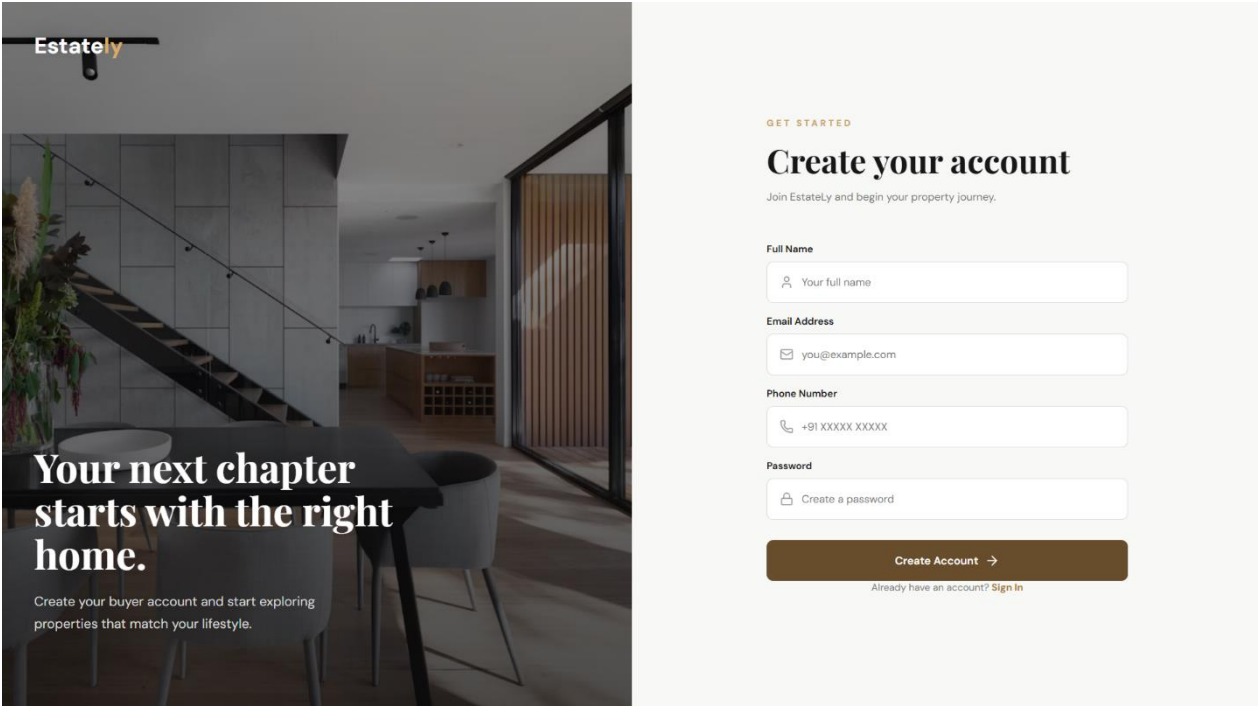
Sign in →

OR

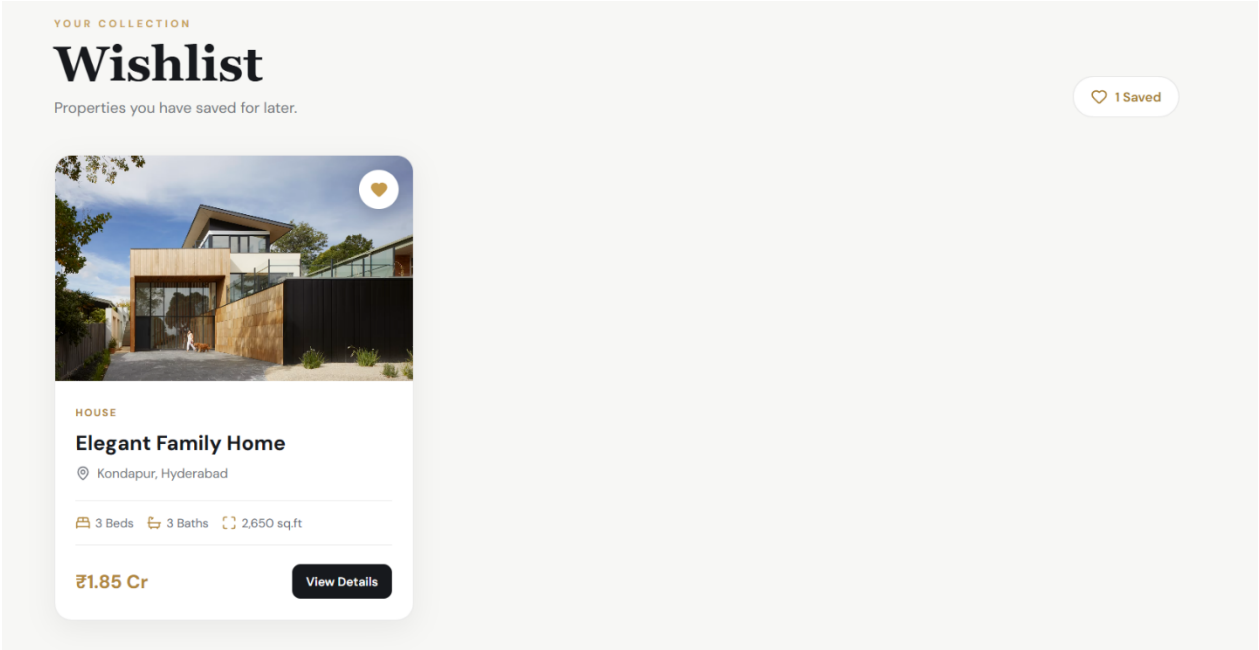
[Don't have an account? Create Account](#)

Page 32

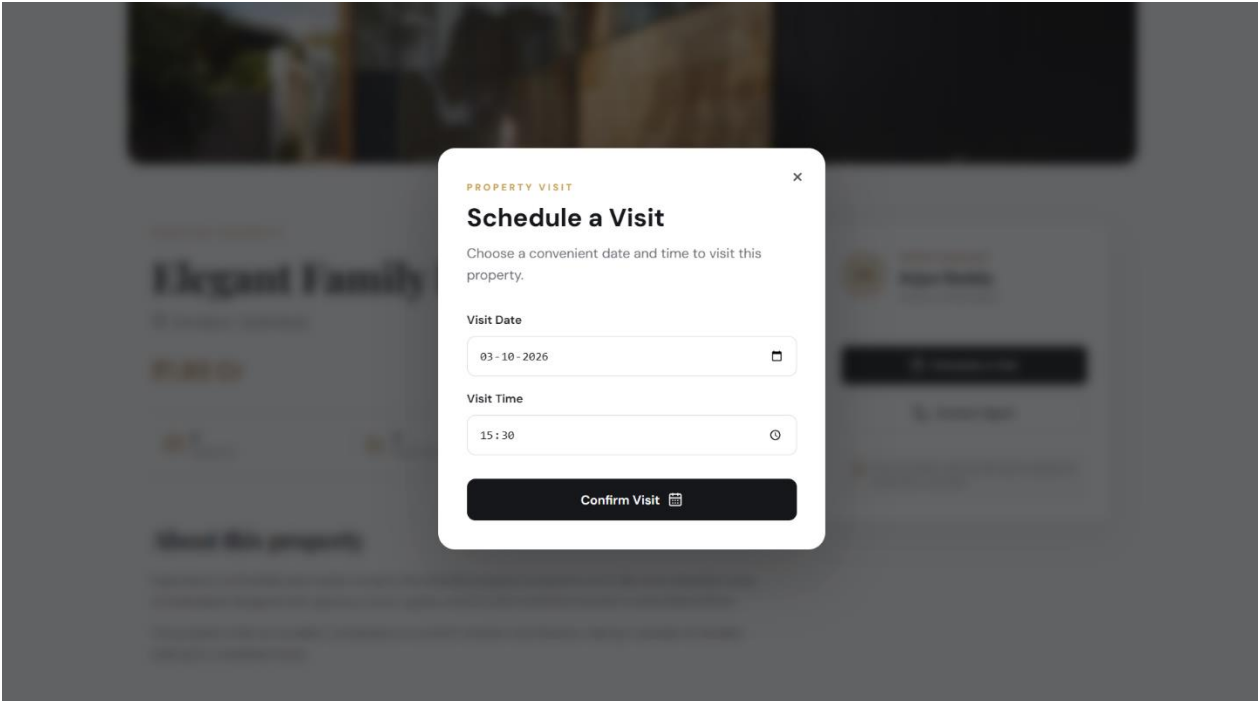
5. Register Page



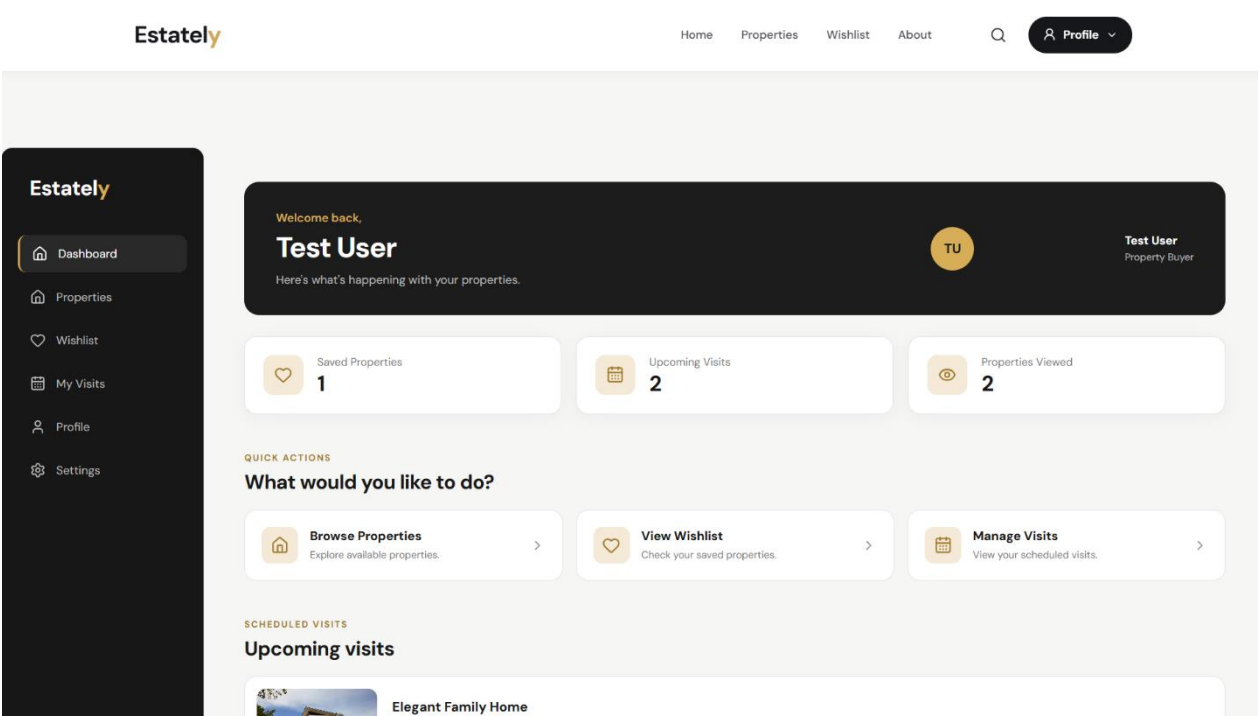
6. Wishlist Page



7. Schedule Visit Interface



8. Dashboard



Browse Properties

Explore available properties.

View Wishlist

Check your saved properties.

Manage Visits

View your scheduled visits.

Dashboard

Properties

Wishlist

My Visits

Profile

Settings

SCHEDULED VISITS

Upcoming visits

Elegant Family Home

Demo Family Home, Kondapur, Hyderabad

2 Oct 2026

12:25 pm

Pending

LOCATION

Demo Family Home, Kondapur, Hyderabad

VISIT DATE

2 Oct 2026

VISIT TIME

12:25 pm

Open in Google Maps

Use ctrl + scroll to zoom the map

Keyboard shortcuts Map data ©2026 Terms Report a map error

Elegant Family Home